



ARQUITETURA DE SISTEMAS

AULA 10 - ARQUITETURA SPACE-BASED



DESAFIO DE ESCALABILIDADE NAS ARQUITETURAS TRADICIONAIS

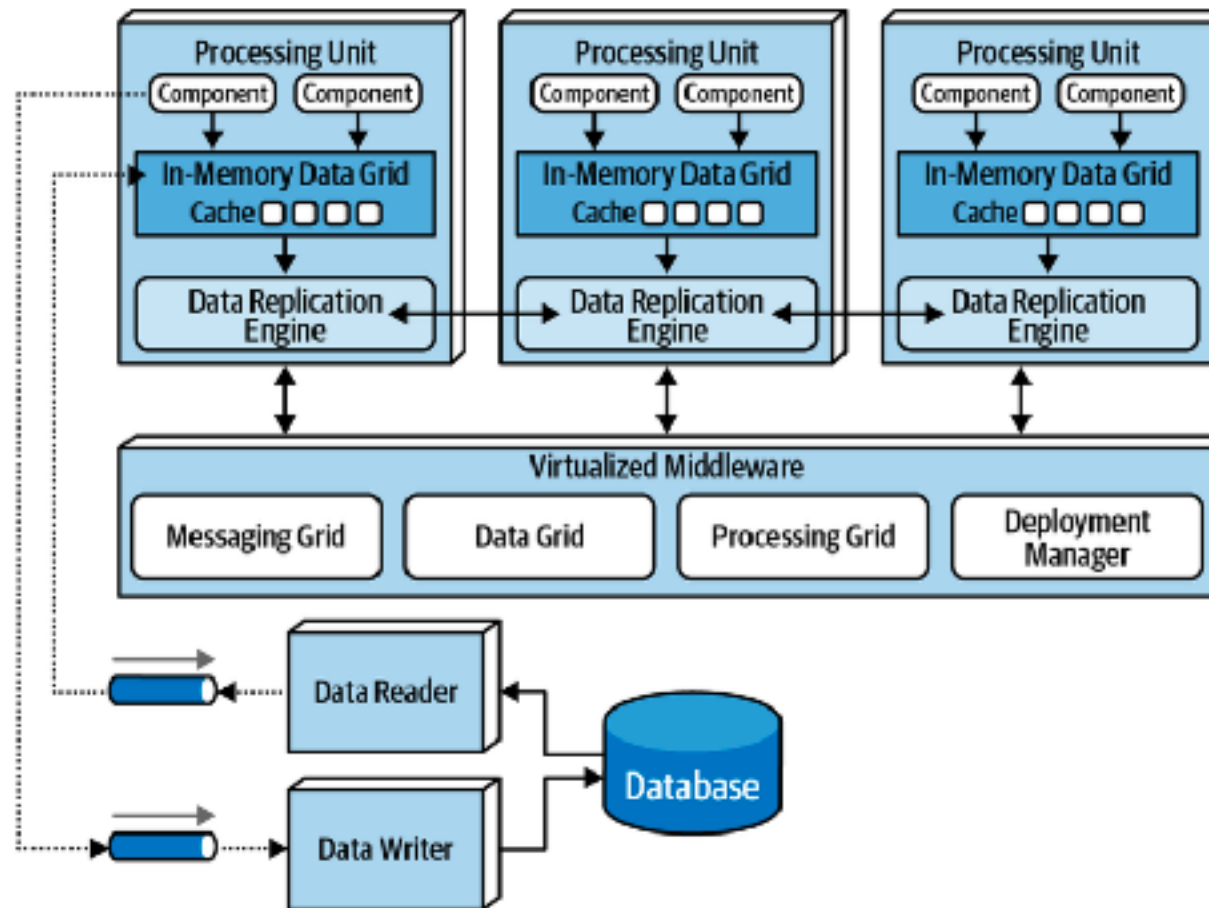
- ❑ Topologia Clássica em Três Camadas: Aplicações web típicas possuem camadas de servidor web, servidor de aplicação e banco de dados. É relativamente fácil escalar horizontalmente os servidores web e de aplicação, mas o banco de dados centralizado torna-se um gargalo conforme o número de usuários e transações cresce.
- ❑ Limitações de Escalabilidade do BD: A capacidade do sistema geralmente fica limitada pela performance do banco de dados, que é o componente mais difícil de escalar. Mesmo com caching e clustering, o BD costuma ser o fator limitante de throughput em altas cargas de trabalho, pois acessar disco ou coordenação de transações distribuídas impõe latência significativa.
- ❑ Forma de Pirâmide Invertida: Com o aumento da carga, podemos adicionar muitas instâncias na camada web e na camada de aplicação, mas todas convergem em um único banco de dados. Isso resulta em uma topologia de carga semelhante a uma pirâmide invertida. Larga na ponta (várias aplicações) e estreita na base (único BD), que eventualmente leva a degradação de desempenho conforme milhares de requisições concorrentes disputam acesso ao mesmo repositório de dados.



SPACE-BASED ARCHITECTURE



SPACE-BASED ARCHITECTURE - TOPOLOGIA



Topologia básica da Space-Based Architecture. A aplicação é composta de múltiplas Processing Units (PUs) idênticas, cada uma com lógica de negócio e um cache de dados em memória. Um middleware distribuído (com componentes como Messaging Grid e Data Grid) direciona as requisições às PUs disponíveis e mantém os dados replicados entre elas, eliminando a necessidade de acesso ao banco de dados durante as transações online. Esse estilo prioriza escalabilidade horizontal e alta disponibilidade.

SPACE-BASED ARCHITECTURE - TOPOLOGIA

- ❑ Eliminação do BD do caminho crítico: No Space-Based Architecture, em vez de acessar um banco de dados central a cada operação, as transações ocorrem inteiramente em memória. Os dados necessários ao processamento estão pré-carregados nas próprias instâncias da aplicação, permitindo throughput muito maior em cenários de alta concorrência, já que praticamente não há I/O de disco no fluxo transacional.
- ❑ Unidades de Processamento (Processing Units): A lógica da aplicação é implantada em múltiplas instâncias autônomas chamadas Processing Units. Cada PU contém componentes de negócio junto com um cache de dados em memória exclusivo. Todas as PUs que compõem o sistema são idênticas e independentes entre si, mas seus caches mantêm-se sincronizados. Ou seja, uma atualização de dado em uma PU é replicada para as demais. Assim, cada unidade possui uma visão atualizada do estado necessário, possibilitando escalabilidade através da clonagem da aplicação em vários nós.



SPACE-BASED ARCHITECTURE - TOPOLOGIA

- ❑ Middleware Virtualizado: Uma camada de middleware distribuído gerencia a coordenação entre as PUs. Esse middleware inclui um Messaging Grid (que recebe as requisições dos clientes e as distribui para uma das instâncias disponíveis, balanceando a carga) e um Data Grid (que cuida da replicação e consistência dos dados em memória entre todas as PUs). Diferentemente de uma orquestração centralizada, aqui a inteligência está distribuída: não há um ponto único de processamento de lógica. Cada PU processa as requisições independentemente, com o middleware garantindo que todas tenham os dados necessários atualizados.



SPACE-BASED - ESCALABILIDADE E ELASTICIDADE

- ❑ Escalabilidade Horizontal: Esse estilo suporta aumentos de carga de forma quase linear. Para atender mais usuários ou throughput, basta adicionar mais Processing Units ao “espaço” de processamento. Como não existe dependência de um servidor único de dados durante a execução, o acréscimo de novas instâncias tende a aumentar proporcionalmente a capacidade do sistema. Em outras palavras, dobrar o número de PUs pode dobrar a quantidade de transações processadas por segundo, até o limite físico de rede ou CPU.
- ❑ Elasticidade sob Demanda: Space-Based Architecture lida bem com picos repentinos de uso, oferecendo suporte nativo à elasticidade. Um componente de gerenciamento (por exemplo, um Deployment Manager) pode monitorar a carga do sistema e iniciar automaticamente novas instâncias de PUs quando detecta aumento de demanda, ou desligá-las quando não são mais necessárias. Assim, antes de um grande evento (por exemplo, vendas relâmpago ou black friday), o sistema pode provisionar instâncias extras para absorver o pico, garantindo desempenho consistente e uso eficiente de recursos.



SPACE-BASED - ESCALABILIDADE E ELASTICIDADE

- ❑ Tolerância a Falhas: Devido à replicação de estado entre instâncias, a falha de uma PU não interrompe o serviço. Outras PUs continuam operando com dados em memória atualizados, pois cada dado importante reside em mais de uma instância simultaneamente (cache replicado). A arquitetura não possui um único ponto de falha facilmente identificável (mesmo o middleware é tipicamente redundante) o que aumenta a disponibilidade. Novas instâncias podem ser iniciadas para substituir PUs falhas, muitas vezes de forma transparente, mantendo o sistema resiliente.



SPACE-BASED -GERENCIAMENTO DE DADOS E PERSISTÊNCIA

- ❑ Persistência Assíncrona (Data Pump): As operações do dia a dia não esperam pela escrita imediata em banco de dados. Em vez disso, as alterações realizadas nas PUs são enviadas a um mecanismo de data pump que faz a persistência de forma assíncrona. Existem componentes Data Writer dedicados a extrair as atualizações do cache em memória e gravá-las periodicamente no banco de dados relacional (por exemplo, a cada poucos segundos ou conforme o volume de mudanças). Essa fila assíncrona de escrita alivia as PUs, permitindo que prossigam com novas transações sem aguardar I/O de disco, e agrega múltiplas operações em escritas otimizadas.



SPACE-BASED -GERENCIAMENTO DE DADOS E PERSISTÊNCIA

- ❑ Data Readers: Analogamente, componentes Data Reader podem atuar alimentando os caches quando uma PU inicia ou quando dados que não estavam em memória são necessários. Eles lêem do banco de dados persistido (por exemplo, dados mestre ou históricos) e populam o data grid em memória para que as PUs passem a ter essas informações localmente. Desse modo, o banco de dados é utilizado como uma fonte de sincronização inicial ou eventual, mas não como parte do caminho de requisição de rotina.
- ❑ Consistência Eventual: Como consequência do modelo de persistência, a arquitetura trabalha com consistência eventual. Isso significa que pode haver um pequeno atraso entre o momento em que uma transação atualiza um dado na PU e o momento em que essa atualização aparece no banco de dados (ou vice-versa, no cache de outra PU). Em geral, esses atrasos são de milissegundos a poucos segundos, controlados pelo mecanismo de replicação e pelo data pump. A aplicação é desenhada para tolerar essa divergência temporária, de forma que a consistência completa dos dados é atingida pouco tempo depois, sem impactar a experiência do usuário durante a transação.



SPACE-BASED - CARACTERÍSTICAS E APLICAÇÕES

- ❑ Alta Performance e Concorrência: A Space-Based Architecture é otimizada para máxima performance em ambientes concorrentes. Ao evitar acessos constantes ao disco e ao distribuir o processamento em vários nós, ela consegue atender um número muito grande de transações por segundo com baixa latência. Características como uso intensivo de memória, eliminação de lock em recursos compartilhados e paralelismo de instâncias tornam esse estilo ideal para cenários de altíssima taxa de requisições simultâneas.
- ❑ Elasticidade para Picos: O modelo baseado em espaço lida muito bem com workloads variáveis. A capacidade de subir ou descer instâncias conforme a necessidade garante que a infraestrutura possa absorver picos súbitos (como um evento de vendas online muito aguardado) sem degradação severa, e depois reduzir de volta para economizar recursos. Essa elasticidade automática é um dos fatores que conferem grande eficiência operacional à arquitetura.



SPACE-BASED - CARACTERÍSTICAS E APLICAÇÕES

- ❑ Complexidade Adicional: Em contrapartida, essa abordagem introduz complexidade no projeto e operação do sistema. É necessário implementar mecanismos sofisticados de replicação de dados, detecção de falhas e monitoramento das PUs e do middleware. Os desenvolvedores precisam considerar cenários de consistência eventual e recuperação de dados em memória após falhas. Portanto, embora a aplicação em si seja desenvolvida de forma modular (cada PU semelhante a uma instância standalone), a arquitetura exige uma infraestrutura robusta (middleware de cache, mensageria, etc.) e uma disciplina rigorosa para manter os dados consistentes.
- ❑ Exemplos de Uso: sistemas de e-commerce ou leilões com altíssimo tráfego (picos de vendas relâmpago), plataformas de venda de ingressos que precisam suportar milhares de usuários concorrentes no momento de abertura de um evento, aplicações de processamento financeiro em tempo real, ou sistemas de gerenciamento de sessões de usuários em larga escala. Nesses casos, os benefícios de desempenho superam a complexidade adicional.



ATÉ A PRÓXIMA AULA